

MI SRC API

Version 1.2

©2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/12/2020
1.1	● Modified API	10/15/2020
1.2	● Add init parameter definition	06/01/2021

目录

REVISION HISTORY	i
目录	ii
1. 概述	1
1.1. 算法说明	1
1.2. 关键词说明	1
1.3. 注意	1
2. API 参考	2
2.1. 功能模块 API	2
2.2. IaaSrc_GetBufferSize	2
2.3. IaaSrc_Init	3
2.4. IaaSrc_Run	4
2.5. IaaSrc_Release	7
3. SRC 数据类型	9
3.1. SRC 模块相关数据类型定义	9
3.2. SrcConversionMode	9
3.3. SRCStructProcess	10
3.4. SrcInSrate	11
3.5. SRC_HANDLE	12
4. 错误码	13

1. 概述

1.1. 算法说明

SRC (Sample Rate Conversion) 重采样，用于对音频流做采样频率转换，以获取不同采样频率的音频流。

1.2. 关键词说明

- 采样频率：每秒从连续讯号中取样的样本数。

1.3. 注意

为方便调试和确认算法效果，需要用户应用自行实现替换算法参数和抓取音频数据的逻辑。

2. API 参考

2.1. 功能模块 API

API 名	功能
IaaSrc_GetBufferSize	获取 Src 算法运行需要的内存大小
IaaSrc_Init	初始化 Src 算法
IaaSrc_Run	Src 算法处理
IaaSrc_Release	释放 Src 算法资源

2.2. IaaSrc_GetBufferSize

➤ 功能

获取 Src 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaSrc_GetBufferSize(SrcConversionMode mode);
```

➤ 形参

参数名称	描述	输入/输出
SrcConversionMode mode	采样率转换类型	输入

➤ 返回值

返回值为 Src 算法运行所需要的内存大小。

➤ 依赖

- 头文件: AudioSRCProcess.h
- 库文件: libSRC_LINUX.so/ libSRC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaSrc_Run](#) 举例部分。

2.3. IaaSrc_Init

➤ 功能

初始化 Src 算法。

➤ 语法

```
SRC_HANDLE IaaSrc_Init(char *workingBufferAddress, SRCStructProcess *src_struct);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Src 算法所使用的内存地址	输入
SRCStructProcess	Src 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioSRCProcess.h
- 库文件: libSRC_LINUX.so/ libSRC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSrc_Run](#) 举例部分。

2.4. IaaSrc_Run

➤ 功能

Src 算法处理。

➤ 语法

```
ALGO_SRC_RET IaaSrc_Run(SRC\_HANDLE handle, short *audio_input, short *audio_output, int*
output_size);
```

➤ 形参

参数名称	描述	输入/输出
handle	Src 算法 handle	输入
audio_input	重采样前的数据指针	输入
audio_output	重采样输出的数据指针	输出
output_size	重采样输出的数据长度，会根据 SRCStructProcess 中设定的 point_number 和 SRC mode 转换的比例而改变。 举例：从 8K 转换到 48K，则 Point number 等比例上升至六倍。	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件：AudioSRCProcess.h
- 库文件：libSRC_LINUX.so/ libSRC_LINUX.a

※ 注意

- audio_output 存放的 buffer 也需要应用程序申请传入。
- Npoints 的取值范围：128, 512, 1024

➤ 举例

```

1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. #include "AudioSRCProcess.h"
5.
6. /* 0:Fixed input file 1:User input file */
7. #define IN_PARAMETER 1
8.
9. int main(int argc, char *argv[])
10. {
11.     SRCStructProcess src_struct;
12.     /******User change section start******/
13.     /*
14.      * Audio file bit wide:The user modifies as needed,
15.      * for example:16, 8
16.      */
17.     int srcFileBitWide = 16;
18.     /*
19.      * The user modifies as needed,for example:1, 2
20.      */
21.     src_struct.channel = 1;
22.     /*
23.      * The user modifies as needed,for example:
24.      * SRATE_8K, SRATE_16K, SRATE_32K, SRATE_48K
25.      */
26.     src_struct.WaveIn_srate = SRATE_8K;
27.     /*
28.      * The user modifies as needed,for example:
29.      * SRC_8k_to_16k, SRC_8k_to_32k, SRC_48k_to_8k ...
30.      */
31.     src_struct.mode = SRC_8k_to_48k;
32.     /*
33.      * The user modifies as needed,for example:
34.      * 256, 512, 1024 and 1536 (Please select among these values)
35.      */
36.     src_struct.point_number = 1536;
37.     /******User change section end******/
38.     SrcConversionMode mode = src_struct.mode;
39.     SRC_HANDLE handle;
40.     ALGO_SRC_RET ret;
41.     int output_size;
42.     unsigned int workingBufferSize;
43.     char *workingBufferAddress = NULL;
44.     int freadBufferSize;
45.     char *freadBuffer = NULL;
46.     int fwriteBufferSize;
47.     char *fwriteBuffer = NULL;
48.     int nBytesRead;
49.     int nPoints = src_struct.point_number;
50.     int MaxFactor = SRATE_48K / SRATE_8K;//The biggest factor
51.     FILE* fpIn; //input file

```



```

52. FILE* fpOut; //output file
53. char src_file[128] = {0};
54. char dst_file[128] = {0};
55.
56. #if IN_PARAMETER
57. if(argc < 3)
58. {
59.     printf("Please enter the correct parameters!\n");
60.     return -1;
61. }
62. sscanf(argv[1], "%s", src_file);
63. sscanf(argv[2], "%s", dst_file);
64. #else
65. sprintf(src_file, "%s", ".8K_16bit_MONO_30s.wav");
66. if(argc < 2)
67. {
68.     printf("Please enter the correct parameters!\n");
69.     return -1;
70. }
71. sscanf(argv[1], "%s", dst_file);
72. #endif
73.
74. fpIn = fopen(src_file, "rb");
75. if(NULL == fpIn)
76. {
77.     printf("fopen in_file failed !\n");
78.     return -1;
79. }
80. printf("fopen in_file success !\n");
81. fpOut = fopen(dst_file, "wb");
82. if(NULL == fpOut)
83. {
84.     printf("fopen out_file failed !\n");
85.     return -1;
86. }
87. printf("fopen out_file success !\n");
88. //(1)IaaSrc_GetBufferSize
89. workingBufferSize = IaaSrc_GetBufferSize(mode);
90. workingBufferAddress = (char *)malloc(sizeof(char) * workingBufferSize);
91. if(NULL == workingBufferAddress)
92. {
93.     printf("malloc SRC workingBuffer failed !\n");
94.     return -1;
95. }
96. printf("malloc SRC workingBuffer success !\n");
97. //(2)IaaSrc_Init
98. handle = IaaSrc_Init(workingBufferAddress, &src_struct);
99. if(NULL == handle)
100. {
101.     printf("SRC:IaaSrc_Init failed !\n");
102.     return -1;
103. }
104. printf("SRC:IaaSrc_Init success !\n");
105. freadBufferSize = src_struct.channel * nPoints * srcFileBitWide / 8;
106. freadBuffer = (char *)malloc(freadBufferSize);
107. if(NULL == freadBuffer)
108. {
109.     printf("malloc freadBuffer failed !\n");
110.     return -1;
111. }

```

```

112. printf("malloc freadBuffer success !\n");
113. fwriteBufferSize = MaxFactor * freadBufferSize;
114. fwriteBuffer = (char *)malloc(fwriteBufferSize);
115. if(NULL == fwriteBuffer)
116. {
117.     printf("malloc fwriteBuffer failed !\n");
118.     return -1;
119. }
120. printf("malloc fwriteBuffer success !\n");
121. #if 0
122.     /*Consider whether the input file has a header*/
123.     fread(freadBuffer, sizeof(char), 44, fpIn); //Remove the 44 bytes header
124. #endif
125. while(1)
126. {
127.     nBytesRead = fread(freadBuffer, 1, freadBufferSize, fpIn);
128.     if(nBytesRead != freadBufferSize)
129.     {
130.         printf("needBytes = %d    nBytesRead = %d\n", freadBufferSize, nBytesRead);
131.         break;
132.     }
133.     //(3)IaaSrc_Run
134.     ret = IaaSrc_Run(handle, (short *)freadBuffer, (short *)fwriteBuffer, &output_size);
135.     if(ret < 0)
136.     {
137.         printf("SRC:IaaSrc_Run failed !\n");
138.         return -1;
139.     }
140.     //printf("ret = %d\n", ret);
141.     fwriteBufferSize = src_struct.channel * output_size * srcFileBitWide / 8;
142.     fwrite(fwriteBuffer, 1, fwriteBufferSize, fpOut);
143. }
144. fclose(fpIn);
145. fclose(fpOut);
146. //(4)IaaSrc_Release
147. ret = IaaSrc_Release(handle);
148. if(ret)
149. {
150.     printf("IaaSrc_Release failed !\n");
151.     return -1;
152. }
153. printf("IaaSrc_Release success !\n");
154. free(workingBufferAddress);
155. free(freadBuffer);
156. free(fwriteBuffer);
157.
158. return 0;
159.}

```

2.5. IaaSrc_Release

➤ 功能

释放 Src 算法资源。

➤ 语法

ALGO_SRC_RET IaaSrc_Release([SRC_HANDLE](#) handle);

➤ 形参

参数名称	描述	输入/输出
handle	Src 算法 handle	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件: AudioSRCProcess.h
- 库文件: libSRC_LINUX.so/ libSRC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSrc_Run](#) 举例部分。

3. SRC 数据类型

3.1. SRC 模块相关数据类型定义

数据类型	定义
SrcConversionMode	Src 算法采样率转换类型
SRCStructProcess	Src 算法初始化参数结构体类型
SrcInSrate	Src 算法的采样率类型
SRC_HANDLE	Src 算法句柄类型

3.2. SrcConversionMode

➤ 说明

定义 Src 算法的采样率转换类型。

➤ 定义

```
typedef enum{
    SRC_8k_to_16k,
    SRC_8k_to_32k,
    SRC_8k_to_48k,
    SRC_16k_to_8k,
    SRC_16k_to_32k,
    SRC_16k_to_48k,
    SRC_32k_to_8k,
    SRC_32k_to_16k,
    SRC_32k_to_48k,
    SRC_48k_to_8k,
    SRC_48k_to_16k,
    SRC_48k_to_32k
}SrcConversionMode;
```

➤ 成员

成员名称	描述
SRC_8k_to_16k	8K 重采样到 16K
SRC_8k_to_32k	8K 重采样到 32K
SRC_8k_to_48k	8K 重采样到 48K
SRC_16k_to_8k	16K 重采样到 8K
SRC_16k_to_32k	16K 重采样到 32K
SRC_16k_to_48k	16K 重采样到 48K
SRC_32k_to_8k	32K 重采样到 8K
SRC_32k_to_16k	32K 重采样到 16K
SRC_32k_to_48k	32K 重采样到 48K
SRC_48k_to_8k	48K 重采样到 8K
SRC_48k_to_16k	48K 重采样到 16K
SRC_48k_to_32k	48K 重采样到 32K

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaSrc_GetBufferSize](#)

3.3. SRCStructProcess

➤ 说明

定义 Src 算法的初始化参数结构体类型。

➤ 定义

```
typedef struct{
    SrcInSrate WaveIn_srate;
    SrcConversionMode mode;
    unsigned int channel;
    unsigned int point_number;
}SRCStructProcess;
```

➤ 成员

成员名称	描述
WaveIn_srate	源数据的采样频率
mode	采样频率转换的模式，请根据源数据采样频率与输出数据采样频率搭配做选择。
channel	源数据的通道数
point_number	输入给 SRC 的点数，IaaSrc_Run 里面的 output_size 会随着此变量与 sampling rate 转换的比例而改变。 如: point_number 为 128，SRC mode 为 SRC_8k_to_48k，则 output_size 为 $128 \times 6 = 768$ ，建议根据所需去设定点数。 可设定值:[256, 512, 1024, 1536, 2880]

※ 注意事项

- 无。

➤ 相关数据类型及接口

[IaaSrc_Init](#)

3.4. SrcInSrate

➤ 说明

定义 Src 算法的采样率类型。

➤ 定义

```
typedef enum{  
    SRATE_8K = 8,  
    SRATE_16K = 16,  
    SRATE_32K = 32,  
    SRATE_48K = 48  
}SrcInSrate;
```

➤ 成员

成员名称	描述
SRATE_8K	8K 采样频率
SRATE_16K	16K 采样频率
SRATE_32K	32K 采样频率
SRATE_48K	48K 采样频率

※ 注意事项

无。

➤ 相关数据类型及接口

[SRCStructProcess](#)

3.5. SRC_HANDLE

➤ 说明

定义 Src 算法的句柄类型。

➤ 定义

```
typedef void* SRC_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaSrc_Init](#)、[IaaSrc_Run](#)、[IaaSrc_Release](#)

4. 错误码

SRC API 错误码如下表所示：

表 4-1 SRC API 错误码

错误码	宏定义	描述
0x00000000	ALGO_SRC_RET_SUCCESS	运行成功
0x10000601	ALGO_SRC_INIT_ERROR	算法尚未初始化
0x10000602	ALGO_SRC_RET_INVALID_MODE	模式参数设置无效
0x10000603	ALGO_SRC_RET_INVALID_HANDLE	HANDLE 无效
0x10000604	ALGO_SRC_RET_INVALID_CHANNEL	通道数不支持
0x10000605	ALGO_SRC_RET_INVALID_POINT_NUMBER	每帧点数不支援
0x10000606	ALGO_SRC_RET_INVALID_SAMPLE_RATE	采样频率不支持
0x10000607	ALGO_SRC_RET_API_CONFLICT	其他 API 正在运行
0x10000608	ALGO_SRC_RET_INVALID_CALLING	呼叫 API 顺序错误